Software Requirements Specification

Traffic Management System Simulation
Version 1.1 | 11 November 2026
Team: Aarya, Vijeth, Vaibhav and Vainik

1 Introduction

This specification defines the implemented standalone C++17 traffic simulation and its acceptance criteria. It covers scenario input, vehicle generation, signal control, movement, routing, traffic management and reporting. The system models traffic; it does not control physical traffic infrastructure. Final implementation evidence is available, while external pipeline execution, human review and administrative submission approval remain pending.

2 Overall description

Scenario designers edit bounded versioned text files. Operators validate, preview, run and trace scenarios from a terminal. Analysts inspect final counters and CSV reports. Developers test the modules through public headers and deterministic scenarios. The application runs on Linux with a C++17 compiler and CMake; automated tests use GoogleTest. Network access is needed only to fetch first-build dependencies, unless an existing GoogleTest source tree is supplied.

Scope excludes live GPS/CCTV, physical controllers, detailed vehicle physics and machine-learning prediction. A graphical interface is optional and is not required for this submission. Directed roads represent two-way connections as two independently configured edges.

3 External interfaces

Supported commands: --help; --validate <file>; --scenario <file> with --show-scenario, --trace, --lane-status and --report <path.csv>. Incidents are scheduled in scenario files. Emergency vehicles submit controlled requests automatically when ready to cross; Simulation::requestPriority is also available to in-process callers. The application runs to its configured duration; Ctrl-C terminates a command. Interactive load/run/status/priority menus from the initial SAD are replaced by this batch CLI.

The version 1 text format includes seed, duration, intersections, roads, lanes, phases, conflicts, adaptive policies, arrivals, streams, batches and incident intervals. See config/README.md for exact grammar and limits. Invalid setup is rejected atomically; CLI exit codes are 0 success, 1 scenario/output error and 2 usage error.

4 Functional requirements

ID and owner	Required behavior	Acceptance evidence
FR-01 / Aarya	Load and validate versioned text scenarios	ScenarioTest.TC_01_LoadsForwardReferencesClassesRoutesAndSignals
FR-02 / Aarya	Generate scheduled, seeded and batch arrivals	VehicleInputTest.UT_A01_AssignsUniqueIdsAndPreservesAttributes
FR-03 / Aarya / Vaibhav	Assign unique IDs and valid attributes and entry-aware routes	ScenarioInputIntegrationTest.RoutesThroughPrescribedEntryDespiteCheaperOtherRoad
FR-04 / Aarya	Enforce FIFO entry capacity and defer arrivals	VehicleInputTest.UT_A04_A05_A06_RetriesPendingBeforeNewArrivalsWithoutChangingIds

ID and owner	Required behavior	Acceptance evidence
FR-05 / Vijeth	Maintain valid signal phases at each controlled intersection	SignalControllerTest.UT_VI01_StartsEachIntersectionInItsConfiguredInitialPhase
FR-06 / Vijeth	Preserve timing, conflict safety and full clearance	SignalControllerTest.UT_VI03_UsesYellowAndAllRedClearanceBeforeNextGreen
FR-07 / Vijeth	Adapt green duration within configured bounds	SignalControllerTest.TC_07_ExtendsGreenForLongQueuesUpToMaximum
FR-08 / Vijeth	Queue safe emergency priority requests	SignalControllerTest.TC_08_PrioritySkipsToRequestedPhaseAfterFullClearance
FR-09 / Vaibhav	Represent valid directed roads, capacities and travel times	RoadNetworkTest.UT_VA01_RegistersConnectedIntersectionsAndExposesRoadProperties
FR-10 / Vaibhav	Move vehicles along routes only under permitted conditions	MovementEngineTest.UT_VA05_EntersRoadAndCrossesOnlyOnPermittedGreen
FR-11 / Vaibhav	Prevent full road and configured downstream lane entry	MovementEngineTest.UT_VA13_FR11_NFR02_DoesNotEnterFullDownstreamLane
FR-12 / Vaibhav	Minimize enabled route travel time with stable ties	RoadNetworkTest.UT_VA02_ReturnsConnectedRouteWithMinimumTravelTime
FR-13 / Vainik	Detect road and queue congestion thresholds	CongestionDetectorTest.UT_VN01_DetectsRoadAtConfiguredOccupancyThreshold
FR-14 / Vainik	Block and restore scheduled incident roads	ScenarioInputIntegrationTest.ParsesAndValidatesScheduledIncidents
FR-15 / Vainik / Vaibhav	Respond to incidents and congestion with safe rerouting	ScenarioInputIntegrationTest.TC_15_ReroutesExistingVehicleAfterLateIncidentAndReplays
FR-16 / Vainik	Measure completion, waiting, peak queues and throughput	ScenarioInputIntegrationTest.TC_16_CountsDeferredWaitingAndPeakQueueBeforeMovement
FR-17 / Vainik	Export final text and CSV and reject output failures	cli_report_write_failure
FR-18 / Shared	Replay identical configuration and seed deterministically	ScenarioInputIntegrationTest.LaneCapacityPreservesSignalsIncidentsReportsAndReset

5 Non functional requirements

ID	Requirement	Verification
NFR-01	Clear public C++ interfaces and controlled Simulation ownership; no uncontrolled mutable globals.	Header and state ownership review; see architecture.md.
NFR-02	Modules expose deterministic methods independently testable with GoogleTest.	Unit suites execute without running the whole simulation.
NFR-03	Run at least four intersections and 500 generated vehicles without hangs; record time and environment.	Measured benchmark in reports/system-validation.md; no invented numeric time threshold.
NFR-04	Reject invalid capacities, timing and network references safely.	Scenario and CLI negative tests, including invalid signal movement references.
NFR-05	Build with standard C++17 and CMake on Linux.	Linux Debug, GCC coverage and analyzer builds; container runtime pending external engine.
NFR-06	Readable CLI commands, errors and metric labels.	Automated CLI checks pass; unfamiliar-user usability review remains pending.
NFR-07	Bound file inputs, validate values and use RAII and safe standard containers.	Malformed input, file limits, references and open/write/close failure tests.

ID	Requirement	Verification
NFR-08	Map all implemented FRs to source paths and executable tests.	docs/traceability.md contains exact tests and source files.
NFR-09	Enable warnings, consistent formatting and clean static analysis.	Pinned clang-format 18.1.8; GCC warnings as errors; Clang analysis.
NFR-10	Accept and record a seed and reproduce traffic/results.	Seed printed and exported; event replay and identical CSV system checks.

6 Security objectives and requirements

ID	Objective
SO-1	Integrity: reject malformed or inconsistent scenario and route data before it reaches movement.
SO-2	Availability: bound file size, line length, duration, stream count and potential generated vehicles.
SO-3	Safety: preserve phase conflict checks, clearance intervals and downstream capacity.

ID	Measurable requirement
SEC-01	Reject malformed values and unknown references, including signal movements, with a nonzero exit and readable error.
SEC-02	Enforce a 1 MiB file limit, 4096-byte lines, duration up to 100000 ticks and at most 100000 potential arrivals.
SEC-03	Use standard containers, strings and RAII; run warnings, formatting and static analysis; reject file output errors, including buffered close failures.
SEC-04	Priority requests may choose only configured safe phases; all yellow/all-red clearance and movement capacity checks remain mandatory.

7 Policy and metric definitions

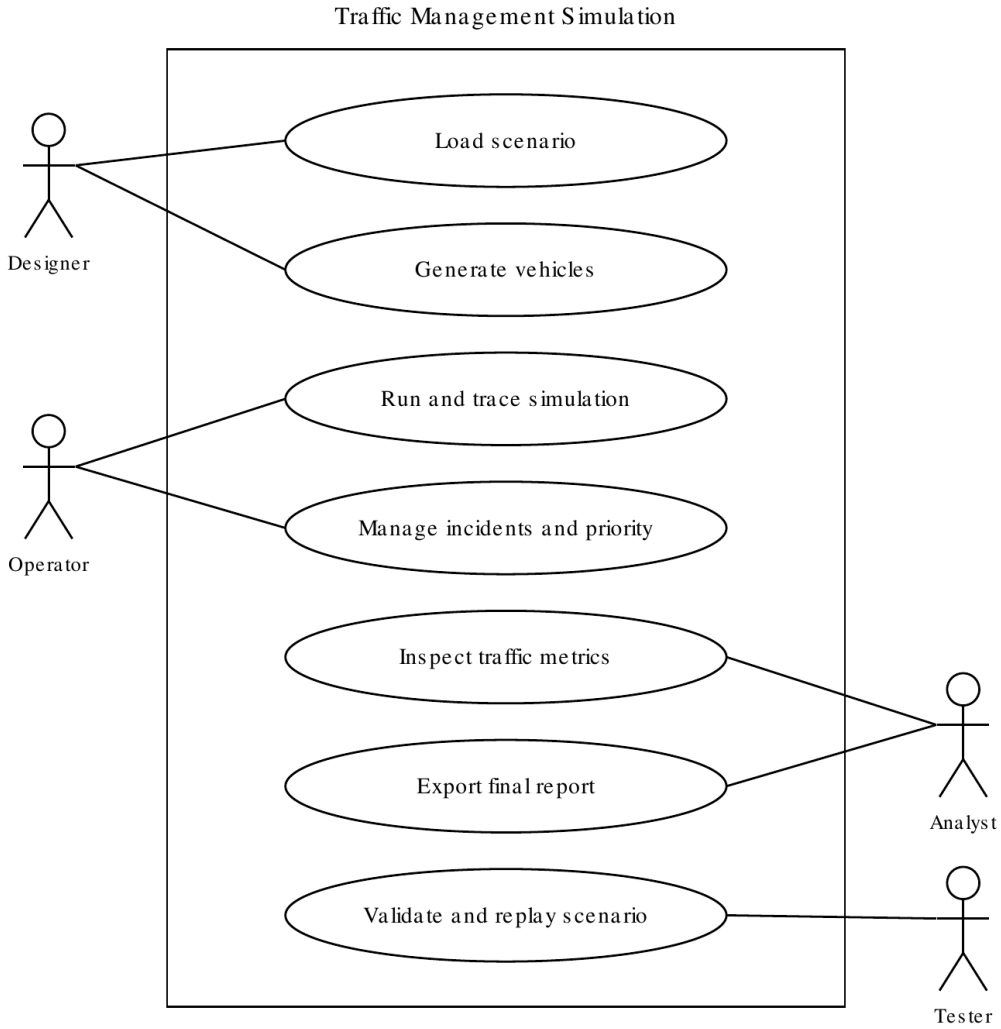
Arrival IDs and file order are stable. Streams run in sorted ID order, using one mt19937_64 draw per active stream per tick. Full queues defer original IDs in FIFO order. Route selection fixes the prescribed entry road and minimizes enabled suffix travel time; equal-cost routes use lexicographic road IDs. Congestion and incidents trigger suffix rerouting while preserving the current road and route index. When no enabled alternative exists, existing vehicles wait safely for restoration. New arrivals with no valid path are marked Unroutable.

Emergency requests are FIFO by target phase and deduplicated. The current green is not truncated; requested phases are selected after the full configured yellow and all-red intervals. Normal cycling resumes when requests are exhausted. Continuous emergency traffic can delay other phases; no maximum normal-traffic delay is specified.

Waiting ticks count each tick an admitted/deferred vehicle remains Waiting at tick end, plus ready-to-cross ticks blocked in transit. Road travel is excluded. Average wait covers completed vehicles only. Peak queue length is the largest individual waiting queue sampled after admission and after movement; deferred backlog is reported separately. Throughput is completed trips divided by processed ticks in [0,currentTick), zero before any tick. Unfinished includes every generated vehicle not completed, including Unroutable. Final congestion is measured after final movement.

8 UML use case diagram

This UML use-case view retains the original actor responsibilities and updates the supported batch operations. Incidents are scheduled through files; priority uses emergency arrivals or the in-process API.



9 Traceability and acceptance

Each FR maps to exact executable evidence in Section 4 and docs/traceability.md. The repository includes separate unit, system, code-coverage and branch-coverage reports under docs/reports. Automated checks establish implemented behavior, not human sign-off. NFR-06 unfamiliar-user review and final team approval remain pending. Docker/Jenkins definitions are supplied; their execution is blocked locally by unavailable services and must be checked on the supported runner.

10 Revision history

Version	Date	Change
1.0	9 October 2026	Original team requirements.
1.1	11 November 2026	Preserve FR/NFR IDs, define security requirements and actual CLI/policies, and map current implementation evidence. Formal acceptance pending.